



Documento de Projeto de Sistema

Midnight Oracle

Vitória, ES

2026

Contribuíram para esta especificação:

Nome	Contato	Contribuições
Alan Herculano Diniz	alan.diniz@edu.ufes.br	Escrita e revisão do documento.
Daniel Fernandes Silva	daniel.silva.22@ufes.br	Escrita e revisão do documento.

Registro de alterações:

<<https://github.com/AHDiniz/MidnightOracle>>

1 Introdução

Este documento apresenta o projeto (*design*) do sistema *Midnight Oracle*, que é um aplicativo web de agregação de feeds RSS, o qual o usuário pode usar para organizar os noticiários e podcasts que ele acompanha em categorias, o que ele pode usar para filtrar e pesquisar em outros momentos.

Além desta introdução, este documento está organizado da seguinte forma: a Seção 2 apresenta a plataforma de software utilizada na implementação do sistema; a Seção 3 apresenta a arquitetura de software; por fim, a Seção 4 apresenta a modelagem dos componentes da arquitetura utilizando a abordagem FrameWeb.

2 Plataforma de Desenvolvimento

Na Tabela 1 são listadas as tecnologias utilizadas no desenvolvimento da ferramenta, bem como o propósito de sua utilização.

Tabela 1 – Plataforma de Desenvolvimento e Tecnologias Utilizadas.

Tecnologia	Versão	Descrição	Propósito
Gleam	1.15.4	Linguagem de Programação Funcional	Compilação de programas tanto para a Beam (máquina virtual do Erlang) quanto para JavaScript
Lustre	5.6.0	Framework para construção de templates HTML	Criação do front-end da aplicação
Wisp	2.2.2	Framework para desenvolvimento web	Roteamento HTTP
Squirrel	4.6.0	Framework type-safe de SQL	Realização de queries em banco de dados
Glow Auth	1.0.1	Framework OAuth 2.0	Autenticação segura de usuários

Na Tabela 2 vemos os softwares que apoiaram o desenvolvimento de documentos e também do código fonte.

Tabela 2 – Softwares de Apoio ao Desenvolvimento do Projeto

Tecnologia	Versão	Descrição	Propósito
TeXstudio	4.9.3	Editor LaTeX	Escrita deste documento.
Visual Studio Code	1.116.0	Editor de Código	Escrita do código fonte.
Mermaid Live Editor	11.14.0	Editor de Diagramas	Criação dos diagramas UML.

3 Arquitetura de Software

A Figura 1 mostra a arquitetura do sistema *Midnight Oracle*.

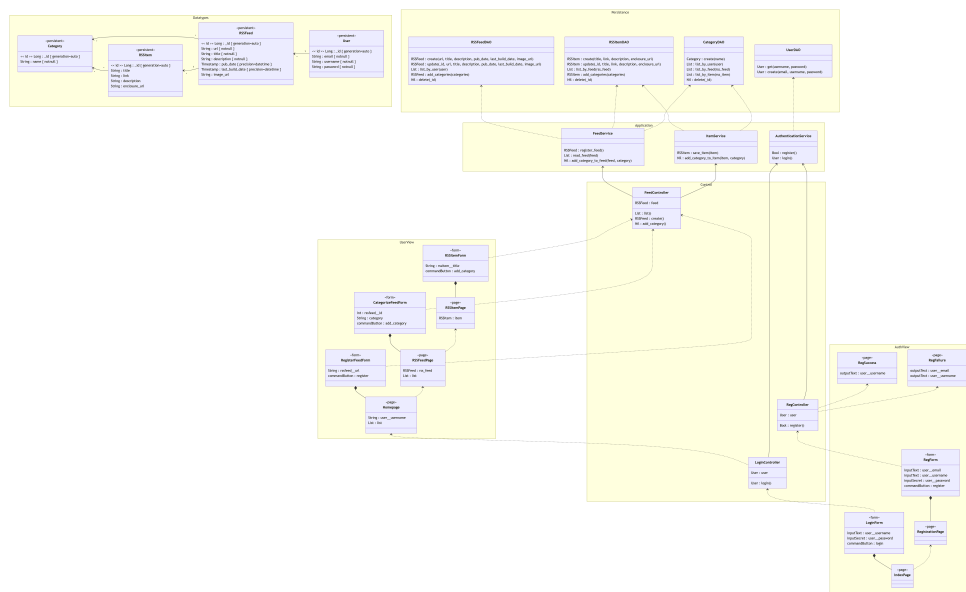


Figura 1 – Arquitetura do sistema.

Podemos ver que a aplicação é dividida em cinco pacotes: um para a definição dos tipos do domínio, um para persistência de dados, dois para controlar a navegação visual da aplicação, e dois para realizar a comunicação entre a navegação e a persistência.

A ideia é que os pacotes *Application* e *Persistence* serão viverem no *back-end*, enquanto os pacotes *Control*, *UserView* e *AuthView* viverem no *front-end*. Enquanto isso, o pacote *Datatypes* será referenciado pelas duas pontas da aplicação, já que representa os dados necessários para a execução da lógica de domínio.

A ideia é que um usuário pode realizar o cadastro na aplicação, o que o permite que ele possa salvar links de feeds RSS na sua conta, e também possa editar as categorias às quais tanto os feeds quanto os itens dos feeds pertencem, por isso estamos salvando esses dados no banco. Isso também permite que, numa suposta versão futura da aplicação, seja possível mostrar o conteúdo de um item do feed dentro de sua página.

4 Modelagem FrameWeb

Midnight Oracle é um sistema Web cuja arquitetura utiliza *frameworks* comuns no desenvolvimento para esta plataforma. Desta forma, o sistema pode ser modelado utilizando a abordagem FrameWeb (Souza, 2020).

A Tabela 3 indica os *frameworks* presentes na arquitetura do sistema que se encaixam em cada uma das categorias de *frameworks* que FrameWeb dá suporte. Em seguida, os modelos FrameWeb são apresentados para cada camada da arquitetura.

Tabela 3 – *Frameworks* da arquitetura do sistema separados por categoria.

Categoria de <i>Framework</i>	<i>Framework</i> Utilizado
Controlador Frontal	Lustre
Injeção de Dependências	Wisp
Mapeamento Objeto/Relacional	Squirrel
Segurança	Glow Auth

4.1 Camada de Negócio

Na camada de negócio, vivem as classes do pacote *Datatypes*, que representam um usuário, que possui nome, email e senha, que pode estar associado a um ou mais feeds RSS, que possuem URL, título, descrição, a data inicial de publicação, a data da última postagem, e um link para uma imagem do feed.

Cada feed pode estar associado a um ou mais itens, que representam cada post do feed, e possuem um título, um link para o post, uma descrição e um link para alguma mídia que esteja ligada ao item, que pode ser, por exemplo, o arquivo de áudio de um episódio de podcast.

Tanto o feed quanto o item podem ter uma ou mais categorias, que possuem o nome dessa categoria, e podem ser extraídas tanto do arquivo do feed RSS quanto ser criadas pelo próprio usuário para que ele possa organizar, filtrar e pesquisar seus feeds.

4.2 Camada de Acesso a Dados

Na camada *Persistence*, vivem as classes do tipo Data Access Object, um para cada tipo da camada de negócio. Nessas classes, além de métodos de criação e apagamento, há também métodos de listagem utilizando critérios específicos, como filtrar por categoria, por feed ou por usuário.

4.3 Camada de Apresentação

Os pacotes restantes lidam com a navegação do *front-end*, ou da comunicação entre o *front-end* com o *back-end*. Existe um pacote para lidar com as páginas de *login* e registro de usuário, e outro para lidar com as páginas para quando o usuário já está logado. Enquanto isso, os pacotes *Control* e *Application* lidam com a comunicação entre as duas pontas, onde o primeiro vive no *front-end* e o segundo *back-end*.

Referências

FOWLER, M. *Patterns of Enterprise Application Architecture*. 1. ed. [S.l.]: Addison-Wesley, 2002. ISBN 9780321127426. Nenhuma citação no texto.

SOUZA, V. E. S. The FrameWeb Approach to Web Engineering: Past, Present and Future. In: ALMEIDA, J. P. A.; GUIZZARDI, G. (Ed.). *Engineering Ontologies and Ontologies for Engineering*. 1. ed. Vitória, ES, Brazil: NEMO, 2020. cap. 8, p. 100–124. ISBN 9781393963035. Available on: <<http://purl.org/nemo/celebratingfalbo>>. Citado na página 4.